Experiment 7

Aim

To identify functional dependencies in a denormalized Sales table and normalize it step-by-step from UNF to 3NF and BCNF. To verify lossless join and dependency preservation using SQL queries.

Source Code

1. Create a denormalized Sales table

```
CREATE TABLE Sales_UNF (
    order_id INT,
    order_date DATE,
    customer_id INT,
    customer_name VARCHAR(50),
    product_ids VARCHAR(100),
    product_names VARCHAR(200),
    quantities VARCHAR(100)
);
INSERT INTO Sales_UNF VALUES

(101, '2026-01-10', 1, 'Dabeer',

 'P1,P2', 'Laptop,Mouse', '1,2'),

(102, '2026-01-11', 2, 'Riya',

 'P2,P3', 'Mouse,Keyboard', '1,1'),

(103, '2026-01-12', 1, 'Aman',

 'P3', 'Keyboard', '2');
```

2. Functional Dependencies

The assumed functional dependencies are:

- order_id → order_date, customer_id
- customer_id → customer_name
- product_id → product_name
- (order_id, product_id) → quantity

The candidate key for the normalized order-details relation is (order_id, product_id).

3. Convert to 1NF

In 1NF, each column must contain a single atomic value.

```
CREATE TABLE Sales_1NF (
    order_id INT,
    order_date DATE,
    customer_id INT,
    customer_name VARCHAR(50),
    product_id VARCHAR(10),
    product_name VARCHAR(50),
    quantity INT,
    PRIMARY KEY (order_id, product_id)
);
INSERT INTO Sales_1NF VALUES
(101, '2026-01-10', 1, ‘Dabeer', 'P1', 'Laptop', 1),
(101, '2026-01-10', 1, 'Amit', 'P2', 'Mouse', 2),
(102, '2026-01-11', 2, 'Riya', 'P2', 'Mouse', 1),
(102, '2026-01-11', 2, 'Riya', 'P3', 'Keyboard', 1),
(103, '2026-01-12', 1, 'Aman', 'P3', 'Keyboard', 2);
```

4. Convert to 2NF

Remove partial dependencies by separating Orders, Products, and Order Details.

```
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    order_date DATE,
    customer_id INT,
    customer_name VARCHAR(50)
);
CREATE TABLE Products (
    product_id VARCHAR(10) PRIMARY KEY,
    product_name VARCHAR(50)
);
CREATE TABLE OrderDetails (
```

```
    order_id INT,
    product_id VARCHAR(10),
    quantity INT,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
INSERT INTO Orders VALUES
(101, '2026-01-10', 1, 'Aman'),
(102, '2026-01-11', 2, 'Riya'),
(103, '2026-01-12', 1, 'Aman');
INSERT INTO Products VALUES
('P1', 'Laptop'),
('P2', 'Mouse'),
('P3', 'Keyboard');
INSERT INTO OrderDetails VALUES
(101, 'P1', 1),
(101, 'P2', 2),
(102, 'P2', 1),
(102, 'P3', 1),
(103, 'P3', 2);
```

5. Convert to 3NF

Remove the transitive dependency order_id → customer_id → customer_name.

```
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50)
);
INSERT INTO Customers VALUES
(1, 'Aman'),
(2, 'Riya');
CREATE TABLE Orders_3NF (
```

```
    order_id INT PRIMARY KEY,
    order_date DATE,
    customer_id INT,
    FOREIGN KEY (customer_id)
        REFERENCES Customers(customer_id)
);
INSERT INTO Orders_3NF VALUES
(101, '2026-01-10', 1),
(102, '2026-01-11', 2),
(103, '2026-01-12', 1);
```

6. Verify the normalized data

```
SELECT o.order_id, o.order_date,
       c.customer_name, p.product_name,
       od.quantity
FROM Orders_3NF o
JOIN Customers c
    ON o.customer_id = c.customer_id
JOIN OrderDetails od
    ON o.order_id = od.order_id
JOIN Products p
    ON od.product_id = p.product_id
ORDER BY o.order_id;
```

7. Verify lossless join and dependency preservation

```
SELECT *
FROM Orders_3NF o
JOIN Customers c
    ON o.customer_id = c.customer_id;
SELECT *
FROM OrderDetails od
JOIN Products p
    ON od.product_id = p.product_id;
```

The joins reconstruct the original order and product details without introducing spurious rows. The decomposition preserves the stated functional dependencies.

8. BCNF verification

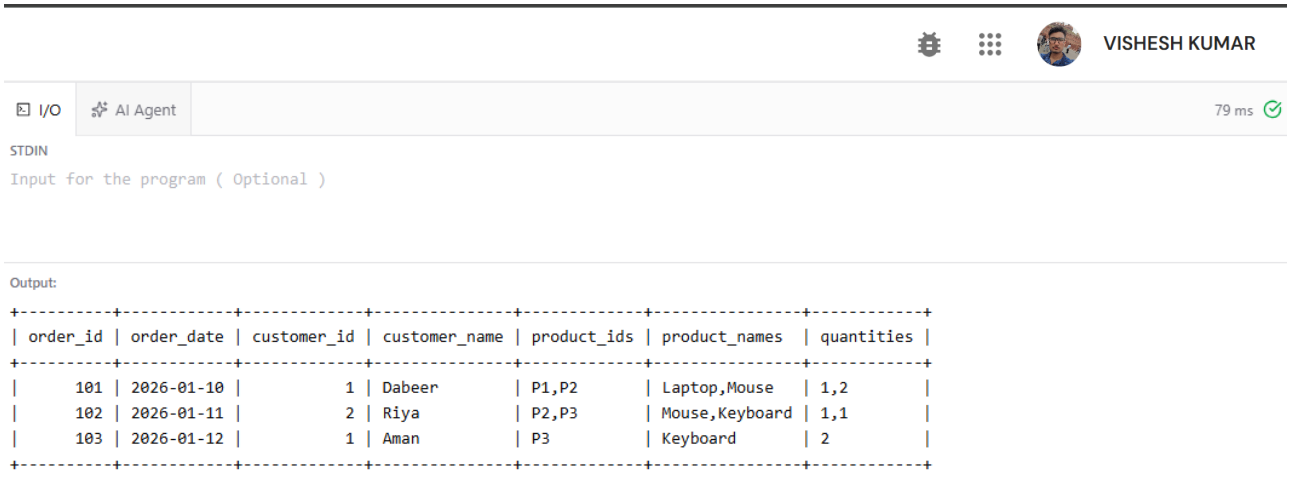
In the final relations, every non-trivial functional dependency has a determinant that is a candidate key:

- Customers: customer_id → customer_name
- Orders_3NF: order_id → order_date, customer_id
- Products: product_id → product_name
- OrderDetails: (order_id, product_id) → quantity

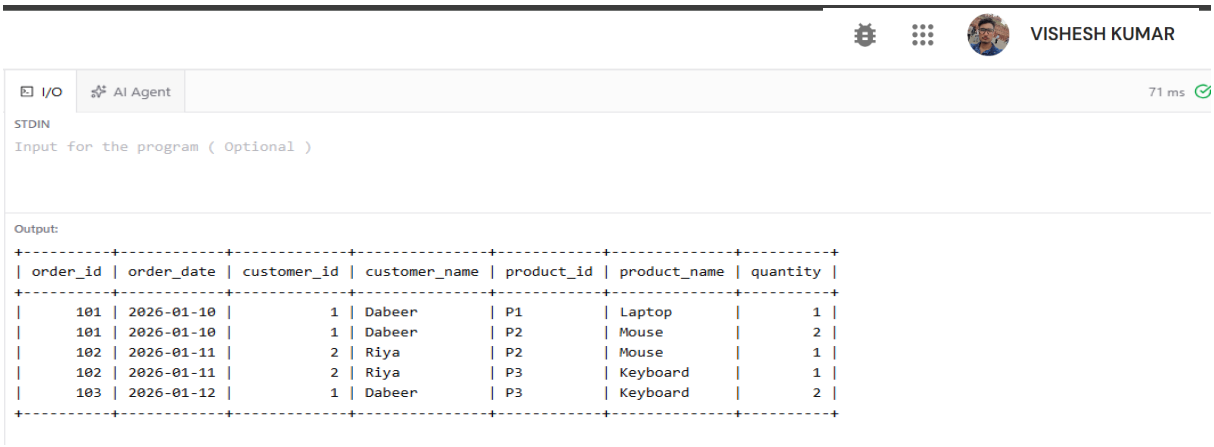
Therefore, these relations satisfy BCNF under the stated dependencies. No further decomposition is required.

Output:

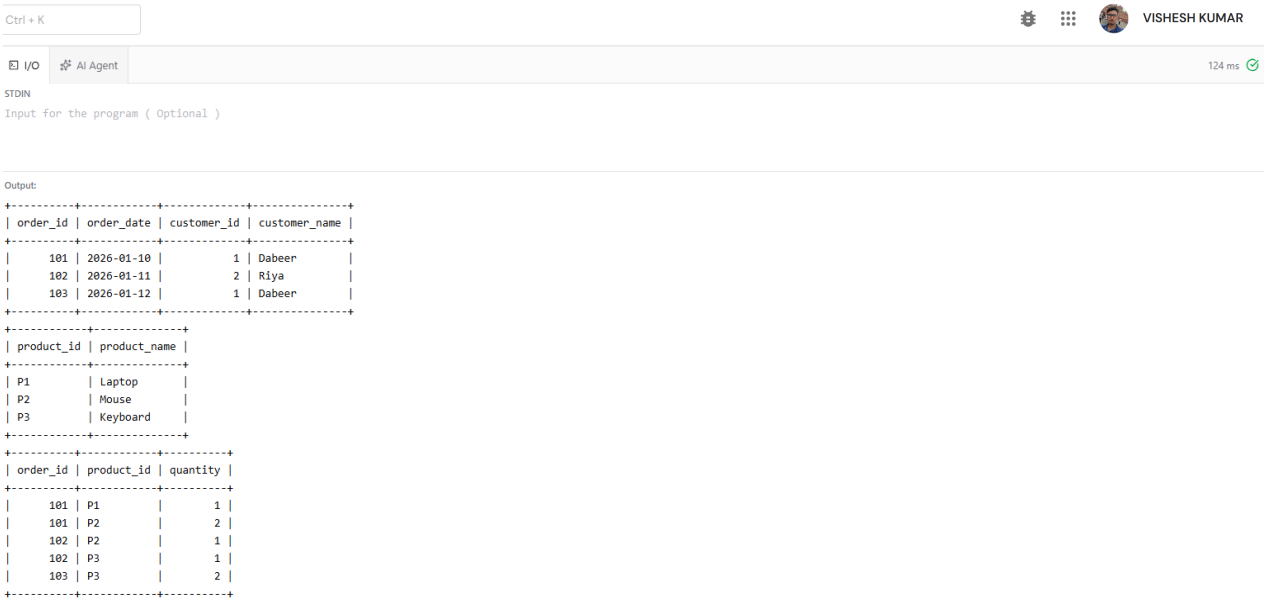
1. Create a denormalized Sales table



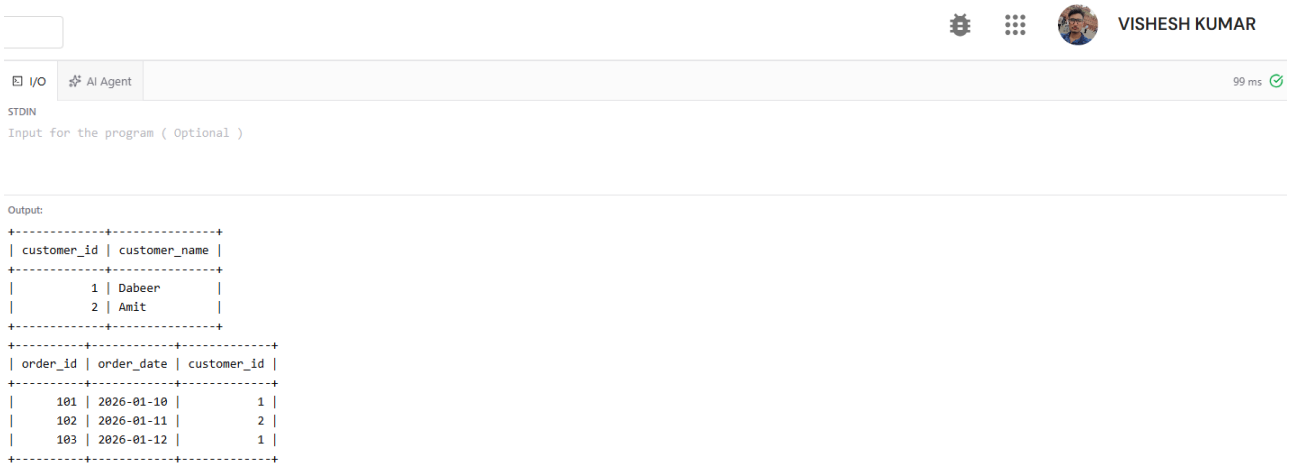
2. Convert to 1NF



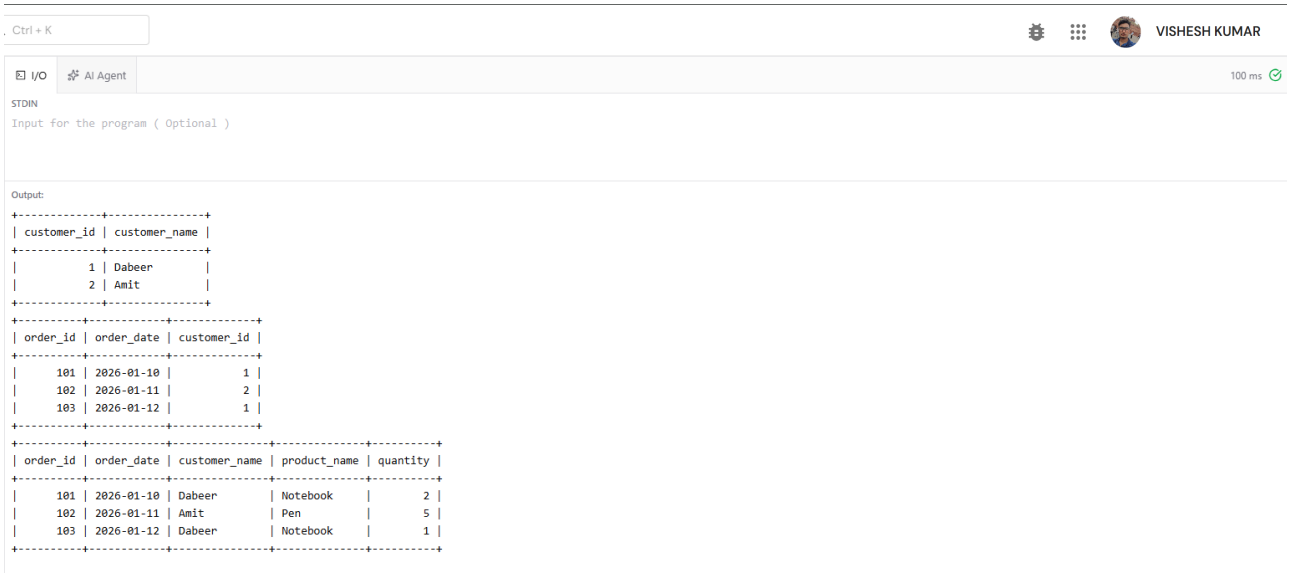
3. Convert to 2NF



4. **Convert to 3NF**



5.**Verify the normalized data**



6. **Verify lossless join and dependency preservation**

